Function approximators (deep learning) & RL

Vincent François-Lavet

June 4, 2026

Outline

Motivation and recall

How can we scale with deep learning as a function approximator?

Different variants of model-free deep RL

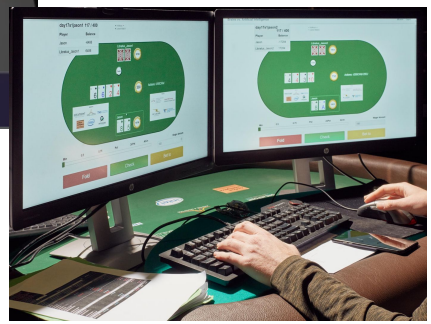
Real-world examples using deep RL

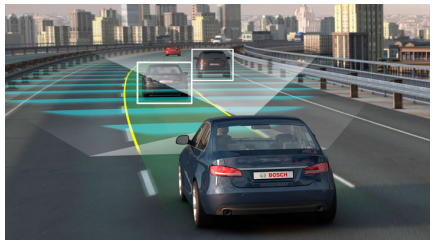
Improving further generalization (links with some upcoming talks)

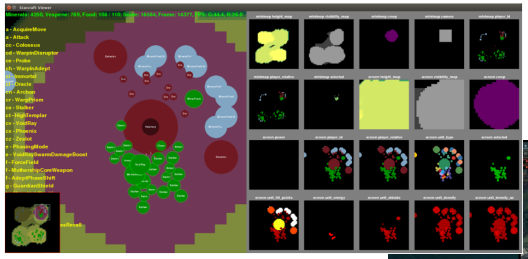
Conclusions

Motivation and recall

Motivation: Overview



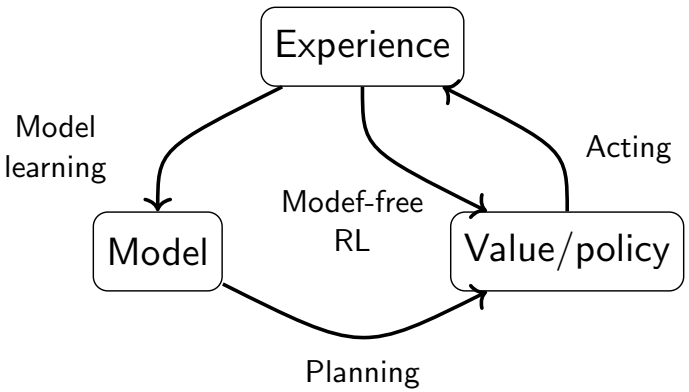


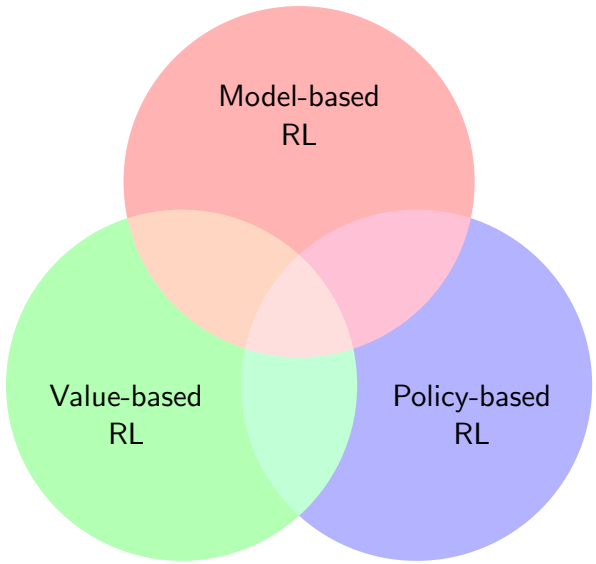


Overview of deep RL

In general, the learning algorithm in RL may include one or more of the following components:

- a value function that provides a prediction of how good is each state or each couple state/action,
- a direct representation of the policy $\pi(s)$ or $\pi(s, a)$, or
- a model of the environment in conjunction with a planning algorithm.



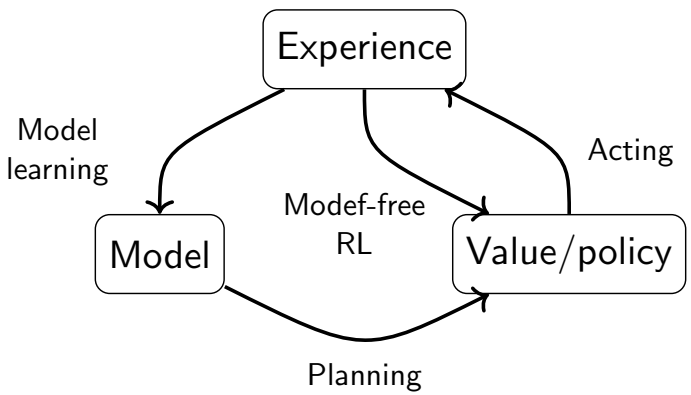


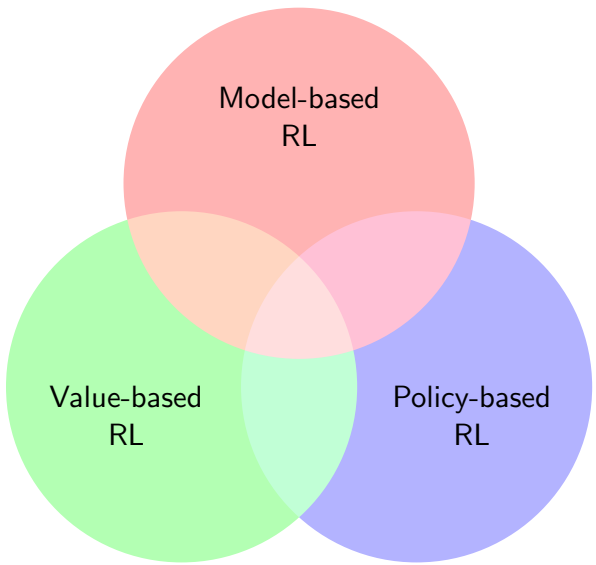
Overview of deep RL

In general, the learning algorithm in RL may include one or more of the following components:

→ a value function that provides a prediction of how good is each state or each couple state/action (main focus),

- a direct representation of the policy $\pi(s)$ or $\pi(s, a)$, or
- a model of the environment in conjunction with a planning algorithm.





Deep learning has brought its generalization capabilities to RL.

Value based methods: recall

In an MDP $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$, the expected return $V^\pi(s) : \mathcal{S} \to \mathbb{R}$ ($\pi \in \Pi$, e.g., $\mathcal{S} \to \mathcal{A}$) is defined such that

$$V^\pi(s) = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s, \pi\right],$$

with $\gamma \in [0, 1)$.

Value based methods: recall

In addition to the V-value function, the Q-value function $Q^{\pi}(s, a) : \mathcal{S} \times A \rightarrow \mathbb{R}$ is defined as follows:

$$Q^{\pi}(s, a) = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^{k} r_{t+k} \mid s_t = s, a_t = a, \pi\right].$$

The particularity of the Q-value function as compared to the V-value function is that the optimal policy can be obtained directly from $Q^{*}(s, a)$:

$$\pi^{*}(s) = \underset{a \in \mathcal{A}}{\operatorname{argmax}}\, Q^{*}(s, a).$$

Value based methods: recall

The Bellman equation that is at the core of value-based learning makes use of the fact that the Q-function can be written in a recursive form:

$$\begin{aligned} Q^{\pi}(s,a) &= \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s, a_t = a, \pi\right] \\ &= \mathbb{E}\left[r_t + \sum_{k=1}^{\infty} \gamma^k r_{t+k} \mid s_t = s, a_t = a, \pi\right] \\ &= \mathbb{E}\left[r_t + \gamma Q^{\pi}(s_{t+1}, a' \sim \pi) \mid s_t = s, a_t = a, \pi\right] \end{aligned}$$

In particular:

$$\begin{aligned} Q^{*}(s,a) &= \mathbb{E}\left[r_t + \gamma Q^{*}(s_{t+1}, a' \sim \pi^*) \mid s_t = s, a_t = a, \pi^*\right] \\ &= \mathbb{E}\left[r_t + \gamma \max_{a' \in \mathcal{A}} Q^{*}(s_{t+1}, a') \mid s_t = s, a_t = a, \pi^*\right] \end{aligned}$$

Convergence Q-learning

Theorem: Given a finite MDP, the Q-learning algorithm given by the update rule

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t[r_t + \gamma \max_{a' \in \mathcal{A}} Q_t(s_{t+1}, a') - Q_t(s_t, a_t)],$$

converges w.p.1 to the optimal Q-function as long as

- $\sum_t \alpha_t = \infty$ and $\sum_t \alpha_t^2 < \infty$, and
- the exploration policy π is such that $P_\pi[a_t = a | s_t = s] > 0, \forall (s, a)$.

Convergence Q-learning

Theorem: Given a finite MDP, the Q-learning algorithm given by the update rule

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t[r_t + \gamma \max_{a' \in \mathcal{A}} Q_t(s_{t+1}, a') - Q_t(s_t, a_t)],$$

converges w.p.1 to the optimal Q-function as long as

- $\sum_t \alpha_t = \infty$ and $\sum_t \alpha_t^2 < \infty$, and
- the exploration policy π is such that $P_\pi[a_t = a | s_t = s] > 0, \forall (s, a)$.

Limitations of tabular approaches

A tabular approach fails for large scale problems due to the curse of dimensionality.

- Robot states with 10 features (e.g. position, speed, angle of joints) discretized into 100 bins $\rightarrow 100^{10} = 10^{20}$ states.
- Chess: $\approx 10^{120}$ states
- Go: $\approx 10^{170}$ statess

Limitations of tabular approaches

A tabular approach fails for large scale problems due to the curse of dimensionality.

- Robot states with 10 features (e.g. position, speed, angle of joints) discretized into 100 bins $\rightarrow 100^{10} = 10^{20}$ states.
- Chess: $\approx 10^{120}$ states
- Go: $\approx 10^{170}$ statess

Three problems

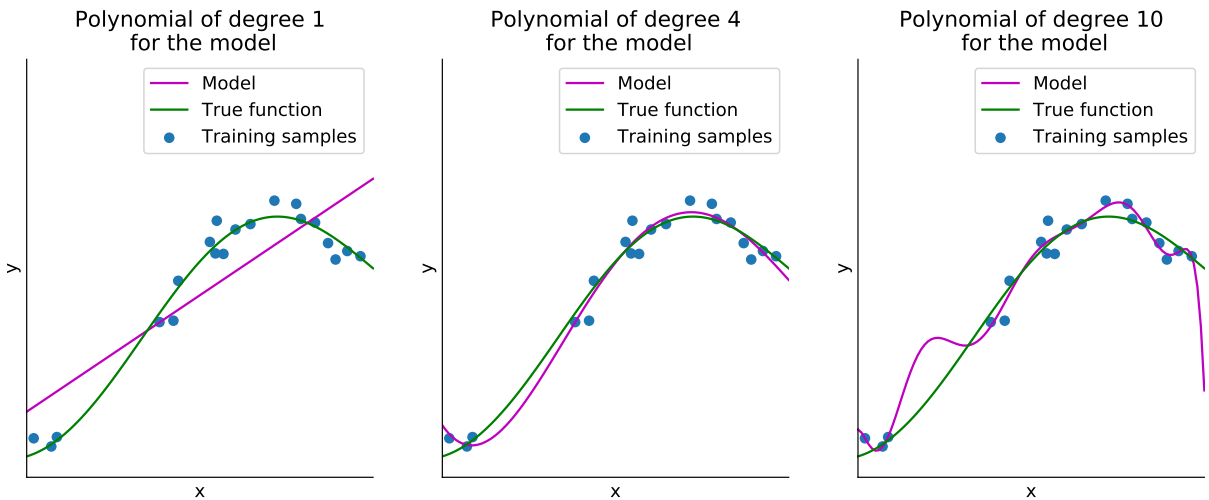
- Memory
- Compute time
- No generalization in the limited data context

Let's move on to function approximators

Function approximators

A function approximator $f : \mathcal{X} \to \mathcal{Y}$ parameterized with $\theta \in \mathbb{R}^{n_\theta}$ takes as input $x \in \mathcal{X}$ and gives as output $y \in \mathcal{Y}$ ($\mathcal{X}$ and $\mathcal{Y}$ depend on the application):

$$y = f(x; \theta). \tag{1}$$



Different types of function approximators

	Flexible	Differentiable
Linear function approximator	✗	✓
SVMs	(✓)	✗
Tree-based approximators	(✓)	✗
⋮	⋮	⋮
Neural networks	✓	✓

Different types of function approximators

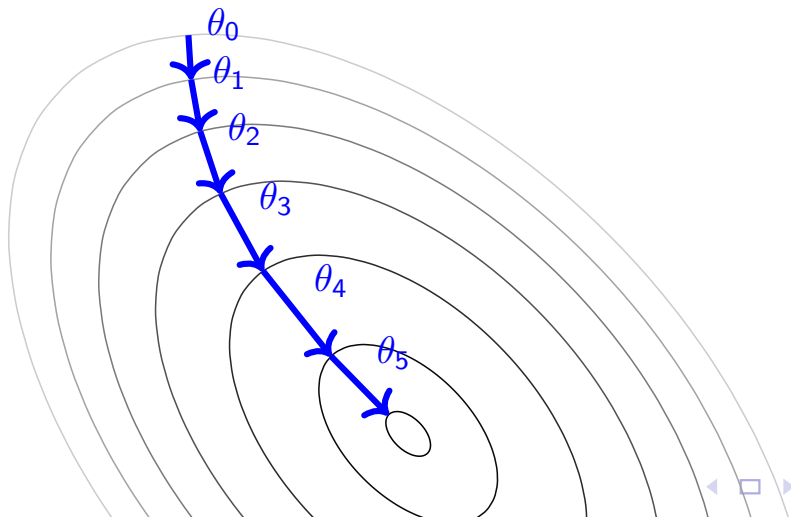
	Flexible	Differentiable
Linear function approximator	✗	✓
SVMs	(✓)	✗
Tree-based approximators	(✓)	✗
⋮	⋮	⋮
Neural networks	✓	✓

Neural networks (with backpropogation of the gradients) has brought its generalization capabilities to RL.

Gradient descent

We need an objective function to be minimized $J(\theta)$ that is a differentiable function of parameters θ.
It starts at an initial point and then repeatedly takes a step opposite to the gradient direction of the function at the current point.

for $k = 0, 1, 2, \ldots$ **do**
$g_k \leftarrow \nabla J(\theta_k)$
$\theta_{k+1} \leftarrow \theta_k - \alpha_k g_k$
end for



How can we scale with deep learning as a function approximator?

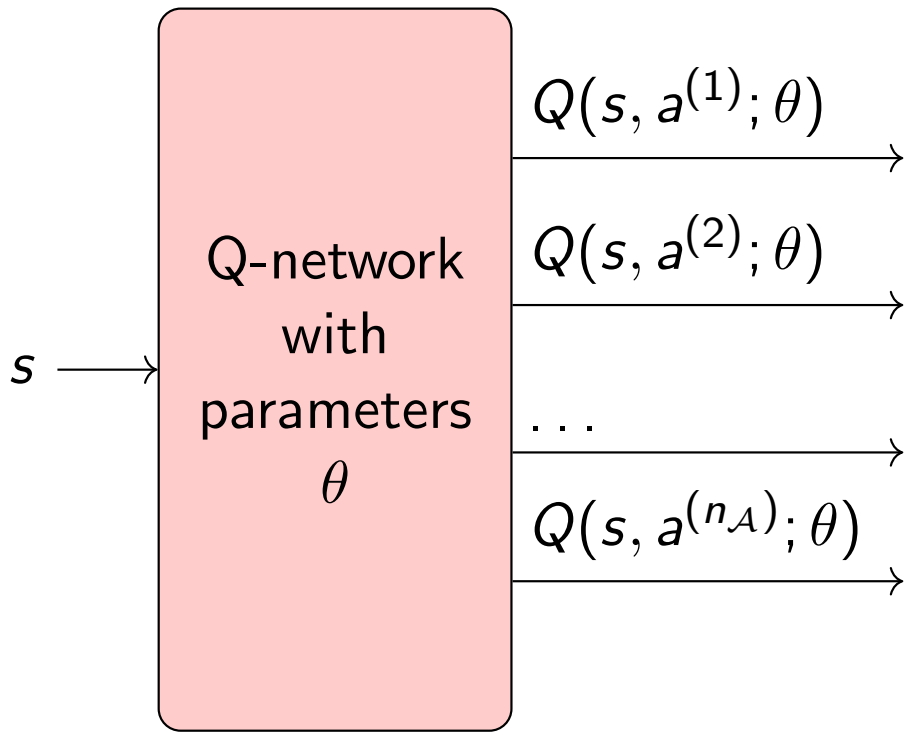
Deep Q-networks

Q-learning with function approximator

We can represent value functions with function approximators and parameters θ:

$$Q(s, a; \theta) \approx Q(s, a)$$

Q-network usual structure (finite number $n_{\mathcal{A}}$ of actions):



Q-learning with function approximator

The parameters θ are updated with gradient descent:

$$\theta := \theta - \alpha \nabla_\theta \underbrace{\left(Q(s, a; \theta) - Y_k^Q \right)^2}_{\text{Objective to be minimized}}$$

with the target

$$Y_k^Q = r + \gamma \max_{a' \in \mathcal{A}} Q(s', a'; \theta_k).$$

Why squared loss is a good candidate for the objective function?

Why squared loss is a good candidate for the objective function?

An Estimator found by minimizing the Mean squared error estimates the distribution's mean.
Said otherwise, for a random variable Y, the minimum of $\mathop{\mathbb{E}}_{y \sim Y}[(c - Y)^2]$ occurs when c is equal to the expected value of Y.

Why squared loss is a good candidate for the objective function?

An Estimator found by minimizing the Mean squared error estimates the distribution's mean.
Said otherwise, for a random variable Y, the minimum of $\mathop{\mathbb{E}}_{y \sim Y}[(c - Y)^2]$ occurs when c is equal to the expected value of Y.

Example: for a given state-action pair (s, a)

- 25% of the time, reward of 1 and a terminal state and
- 75% of the time, a reward of 0 and a terminal state.

The minimum of $\mathbb{E}[(Q(s, a) - Y_k^Q)^2]$ is obtained for Q(s,a)=0.25.

DQN algorithm

For Deep Q-Learning, we can represent value function by deep Q-network with weights θ (instabilities !). In the DQN algorithm:

- Replay memory
- Target network

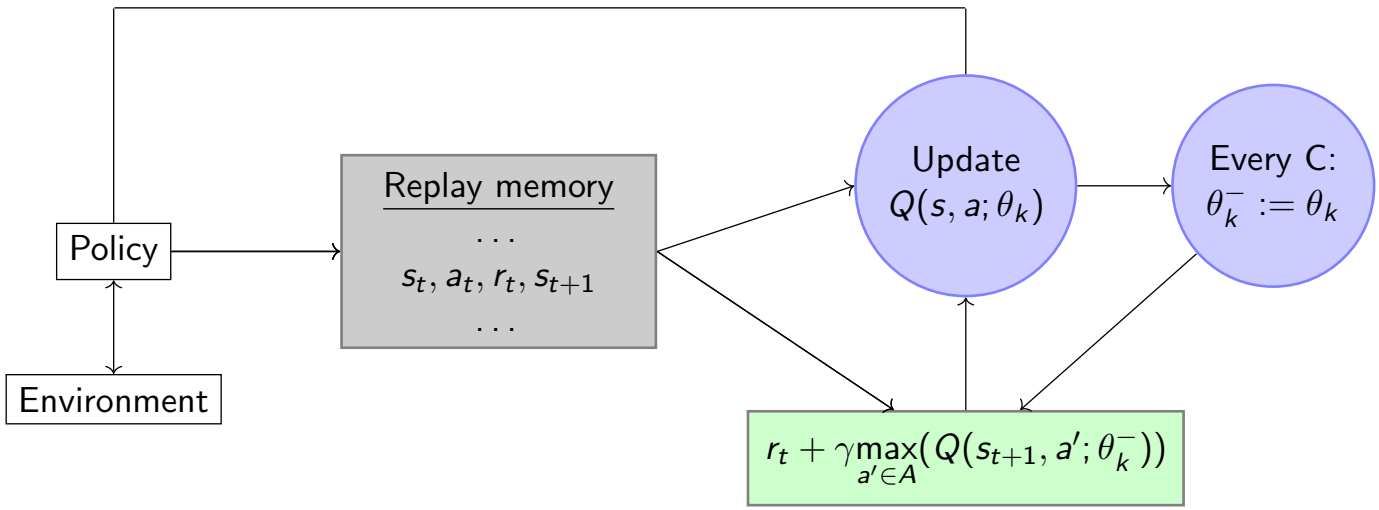


Figure: Sketch of the DQN algorithm. $Q(s, a; \theta_k)$ is initialized to random values (close to 0) everywhere on its domain and the replay memory is initially empty; the target Q-network parameters θ_k^- are only updated every C iterations with the Q-network parameters θ_k and are held fixed between updates; the update uses a mini-batch (e.g., 32 elements) of tuples $< s, a, r, s' >$ taken randomly in the replay memory.

Example: Mountain car

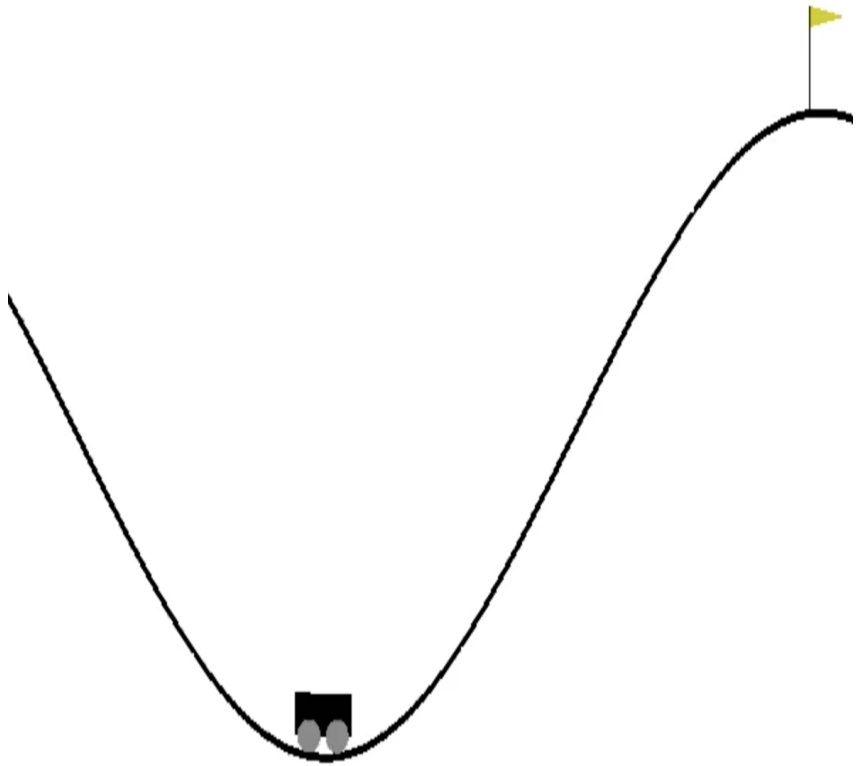


Figure: Mountain car optimal policy

Visualization of Q-values in mountain car

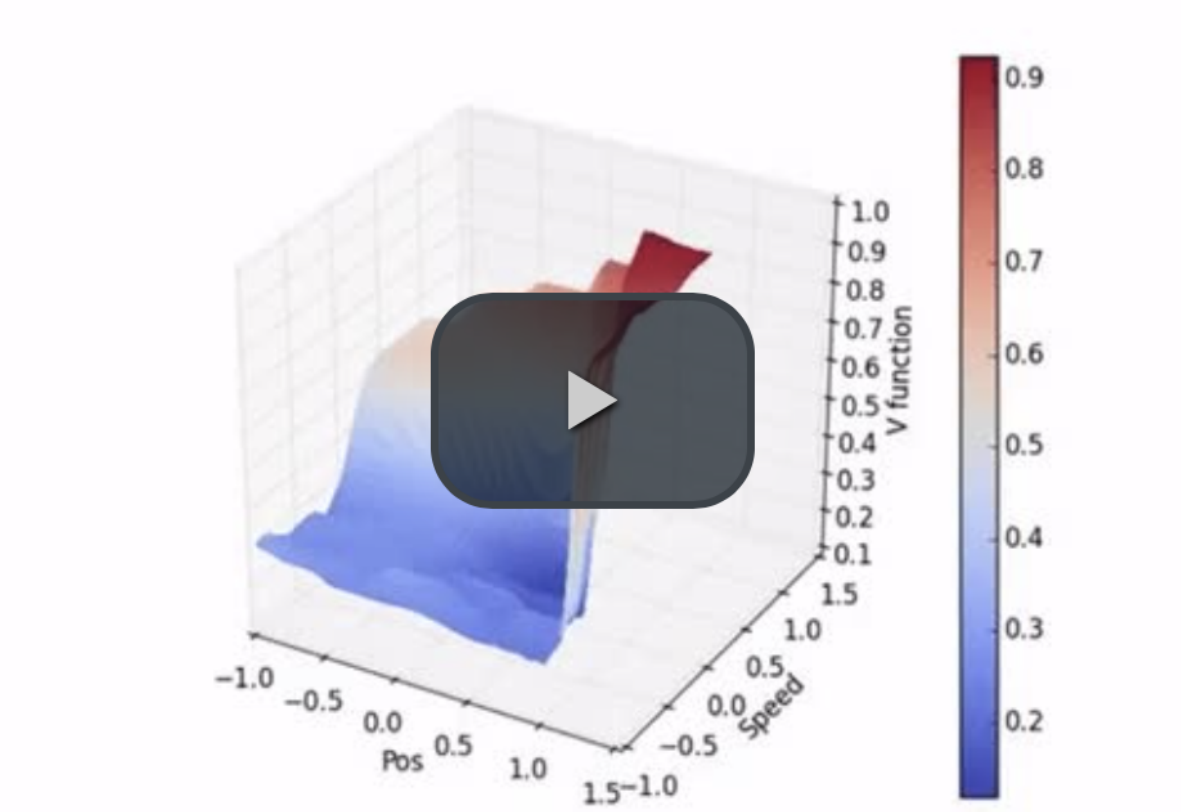


Figure: DQN for mountain car ($V = \max_a Q(s, a)$)

Mountain car

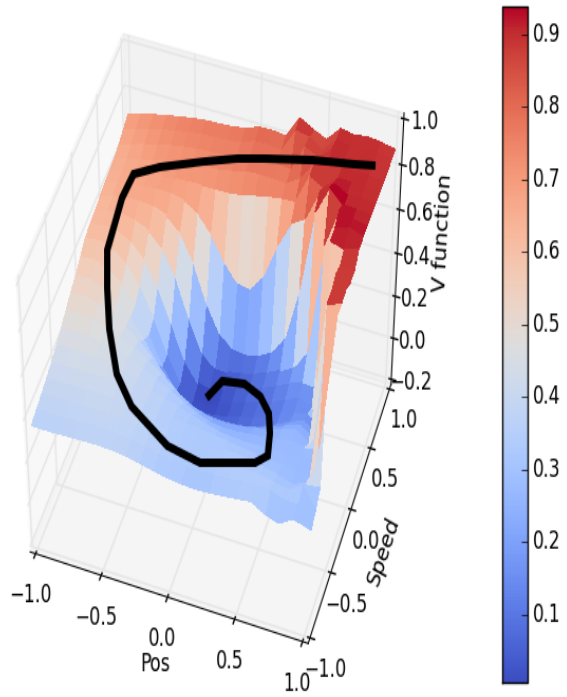


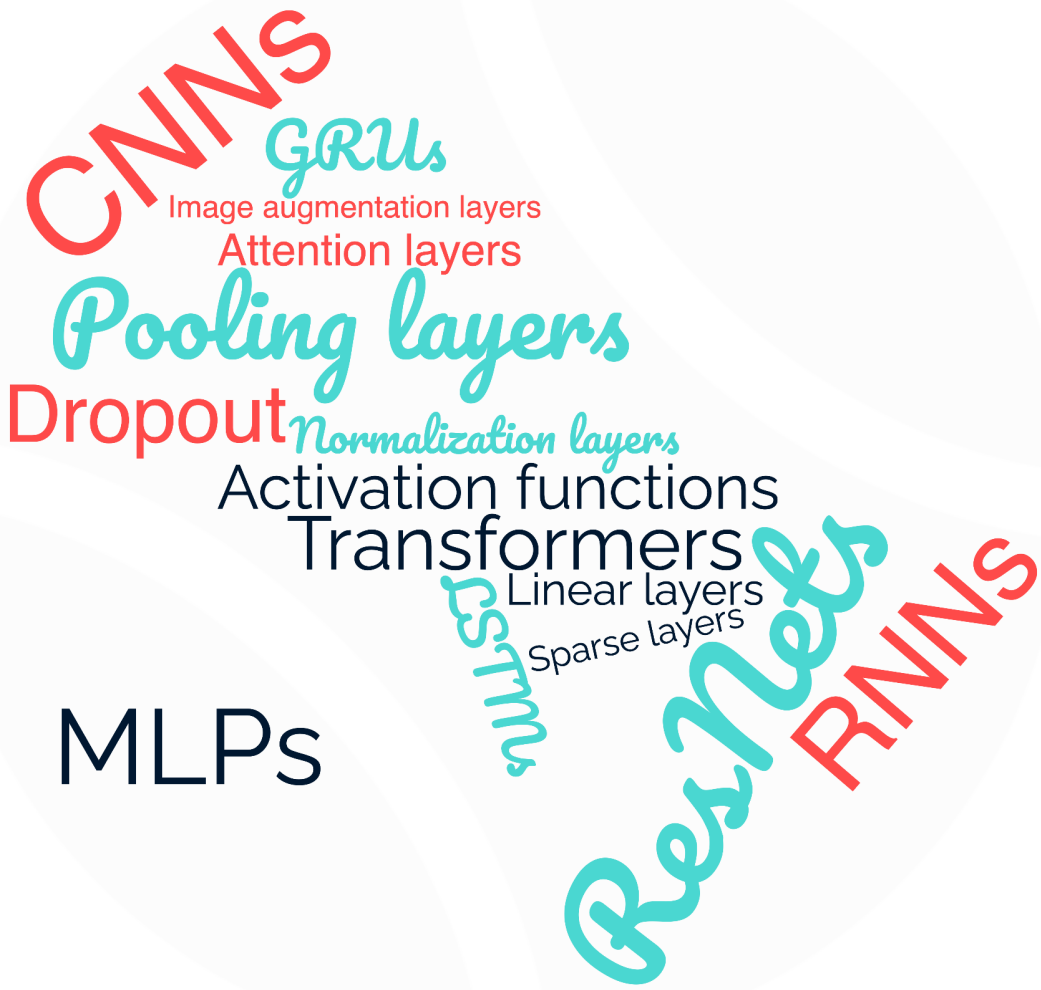
Figure: Application to the mountain car domain: $V = \max_a Q(s, a)$.

Different variants of model-free deep RL

The right deep learning architecture

Deep learning architectures

Similarly to other fields of machine learning, there exists a whole zoo of deep learning modules that can be combined into one deep learning architecture.



CNNs

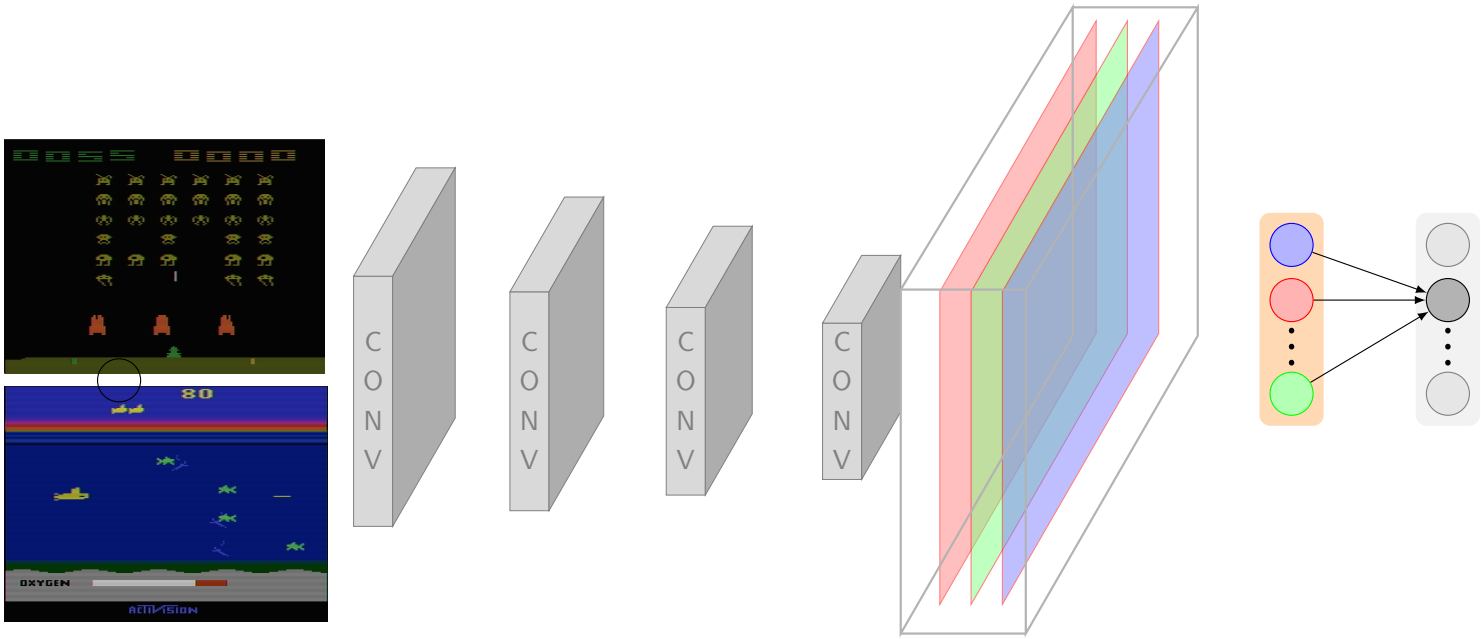


Figure: CNNs are commonly used as first stage of a deep learning architecture when learning from frames, followed by fully connected layers.

RNNs

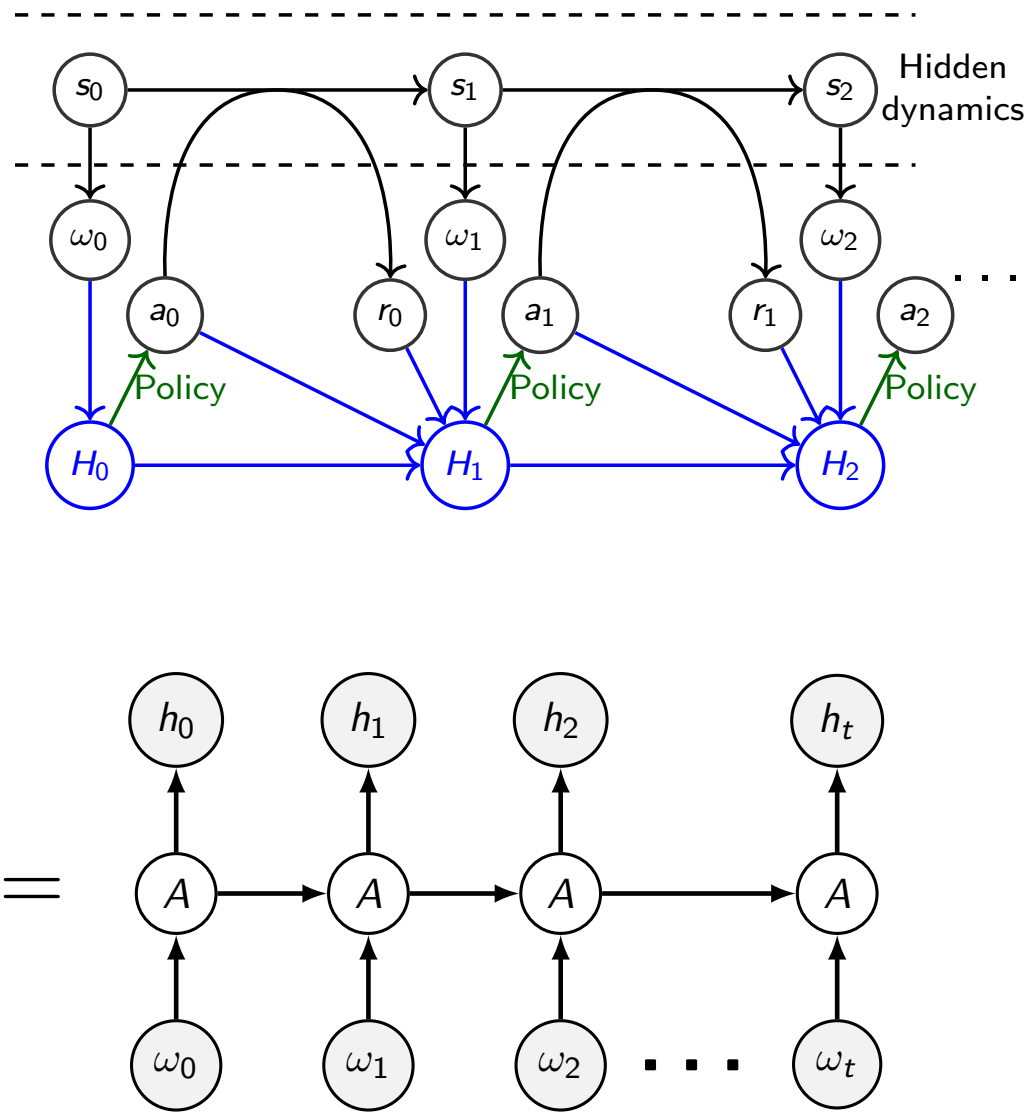


Figure: RNNs are commonly used as first stage of a deep learning architecture when learning from time series, in particular in the context of POMDPs (h_t: hidden layer vector).

Deep learning architectures

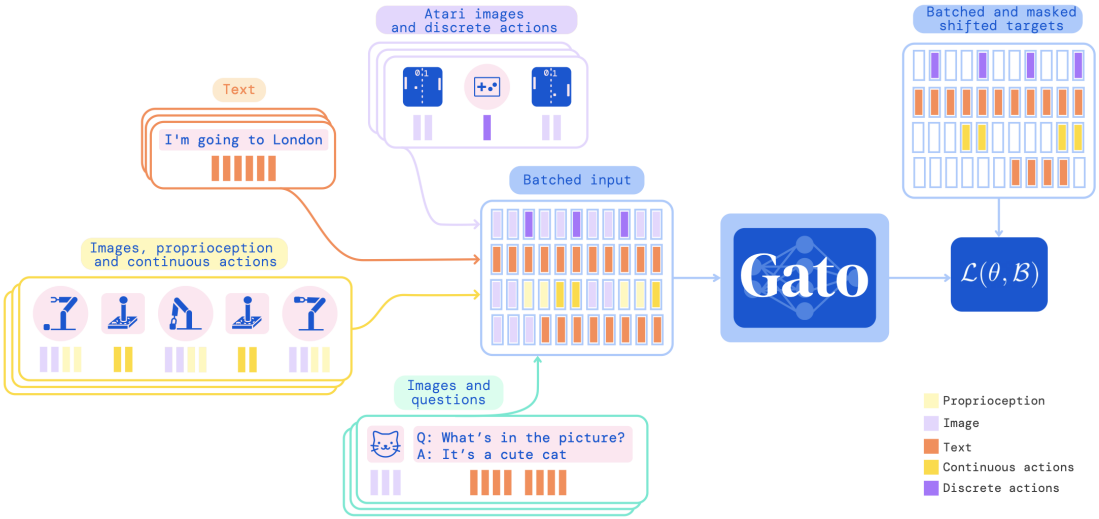


Figure: Transformers architectures have already been used in the context of decision-making, in particular to distill the policies learned over many different domains (source: "A generalist agent" https://arxiv.org/pdf/2205.06175.pdf)

A transformer is a deep learning model that adopts the mechanism of self-attention, differentially weighting the significance of each part of the input data.

Questions so far?

Beyond deep Q learning

- Double DQN
- Multi-step learning
- How to discount deep RL
- Dueling network
- Fighting overfitting and lack of plasticity
- Distributional RL
- Some state-of-the-art results

Double DQN

Double DQN

In Double DQN, or DDQN, the target value Y_k^Q is replaced by

$$Y_k^{DDQN} = r + \gamma Q(s', \underset{a \in \mathcal{A}}{\text{argmax}}\, Q(s', a; \theta_k); \theta_k^-), \tag{2}$$

which leads to less overestimation of the Q-learning values, as well as improved stability, hence improved performance. As compared to DQN, the target network with weights θ_t^- are used for the evaluation of the current greedy action.

Multi-step learning

Multi-step learning

In DQN, the target value used is estimated based on its own value estimate at the next time-step (full bootstrap).

$$Y_k^Q = r + \gamma \max_{a' \in \mathcal{A}} Q(s', a'; \theta_k).$$

A variant uses the n-step target value given by:

$$Y_k^{Q,n} = \sum_{t=0}^{n-1} \gamma^t r_t + \gamma^n \max_{a' \in A} Q(s_n, a'; \theta_k)$$

where $(s_0, a_0, r_0, \cdots, s_{n-1}, a_{n-1}, r_{n-1}, s_n)$ is any trajectory of $n+1$ time steps with $s = s_0$ and $a = a_0$.
Warning: Online data is required for convergence without bias (or other specific techniques)

Dueling network

Dueling network architecture

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$$

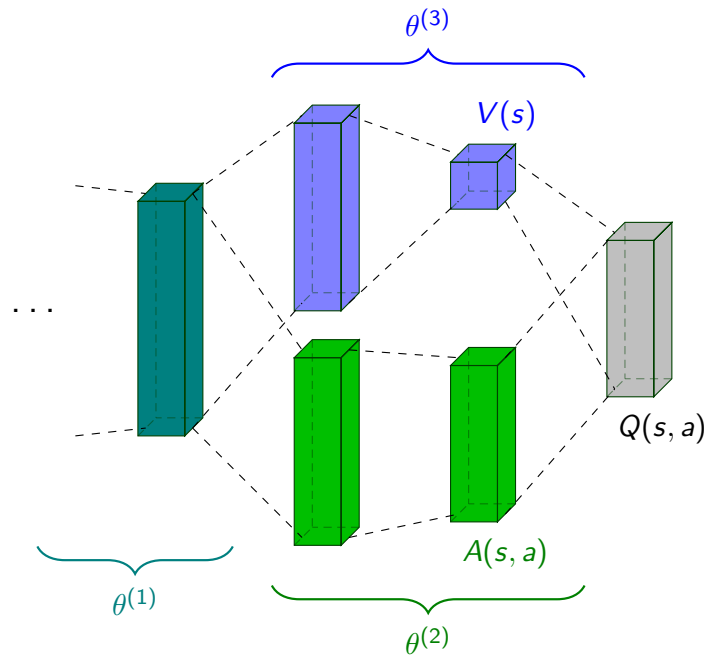


Figure: Illustration of the dueling network architecture.

$$Q(s,a;\theta^{(1)},\theta^{(2)},\theta^{(3)}) = V\left(s;\theta^{(1)},\theta^{(3)}\right) + \left(A\left(s,a;\theta^{(1)},\theta^{(2)}\right) - \max_{a' \in \mathcal{A}} A\left(s,a';\theta^{(1)},\theta^{(2)}\right)\right).$$

Distributional RL

Distributional DQN

The value distribution Z^π is a mapping from state-action pairs to distributions of returns when following policy π. It has an expectation equal to Q^π:

$$Q^\pi(s,a) = \mathbb{E}Z^\pi(s,a).$$

This random return is also described by a recursive equation, but one of a distributional nature:

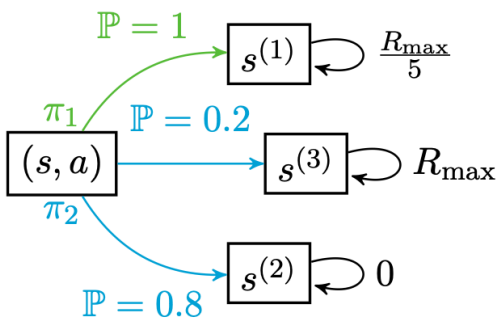
$$Z^\pi(s,a) = R(s,a,S') + \gamma Z^\pi(S',A'),$$

where we use capital letters to emphasize the random nature of the next state-action pair (S',A') and $A' \sim \pi(\cdot|S')$.

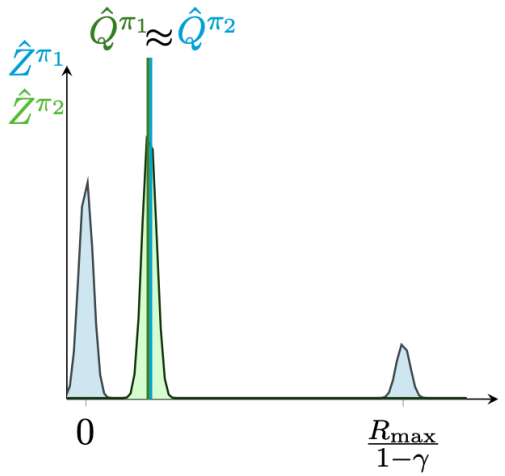
Distributional DQN

It has been shown that such a distributional Bellman equation can be used in practice, with deep learning as the function approximator. This approach has the following advantages:

- It is possible to implement risk-aware behavior.
- It can lead to more performant learning in practice. One of the main elements is that the distributional perspective naturally provides a richer set of training signals than a scalar value function $Q(s, a)$ (effect of *auxiliary tasks*).

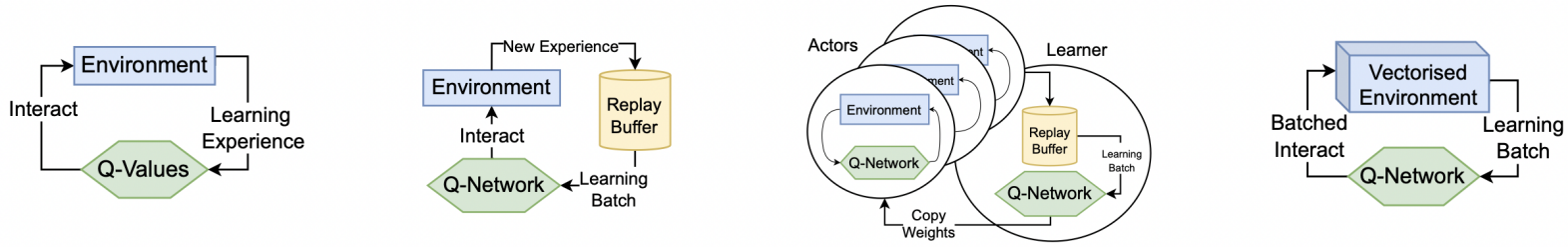


(a) Example MDP.



(b) Sketch (in an idealized version) of the estimate of resulting value distribution $\hat{Z}^{\pi_1}$ and $\hat{Z}^{\pi_2}$ as well as the estimate of the Q-values $\hat{Q}^{\pi_1}, \hat{Q}^{\pi_2}$.

State of the art



(a) Online Q-Learning (b) DQN (c) Distributed DQN (d) PQN

Figure: Evolution of compute architectures in value-based reinforcement learning: from online Q-learning to distributed and parallelized Q-network training.

Source: "Simplifying deep temporal difference learning" (2025), Gallici, Matteo, et al.

State of the art

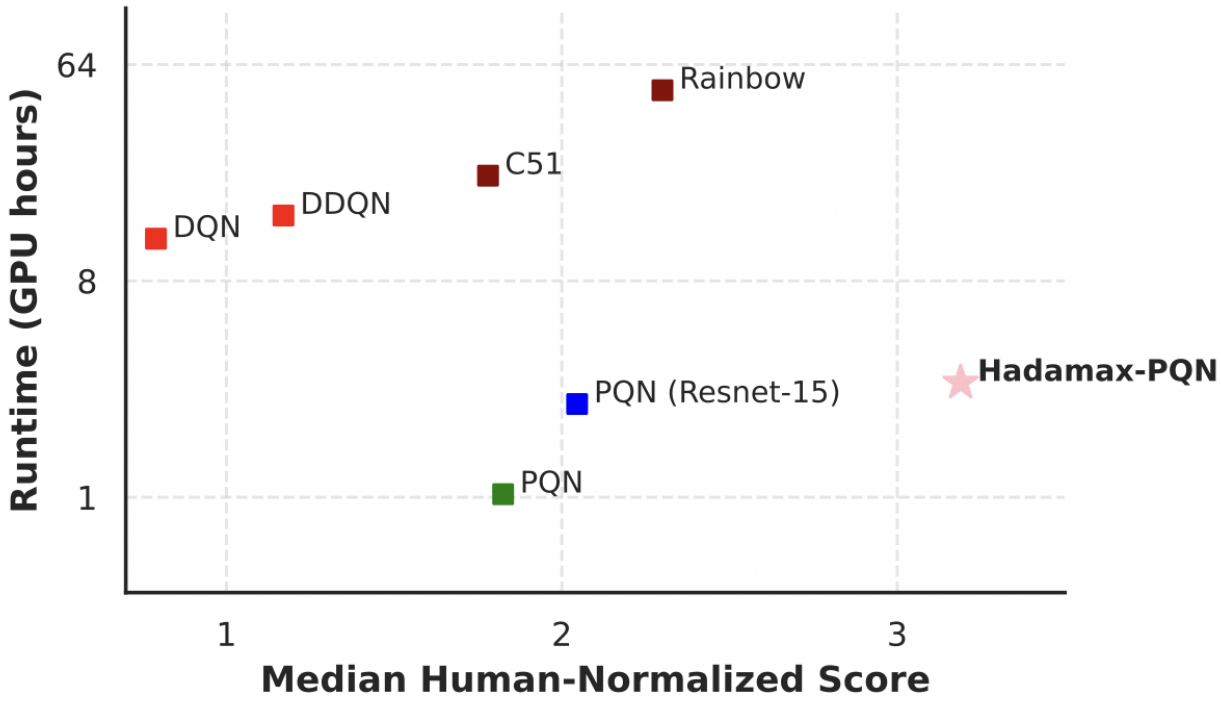


Figure: State of the art algorithms on the Atari benchmark.

Source: "Hadamax Encoding: Elevating Performance in Model-Free Atari" (2025), J. Kooi et al.

Real-world examples using deep RL

Stratospheric Balloon

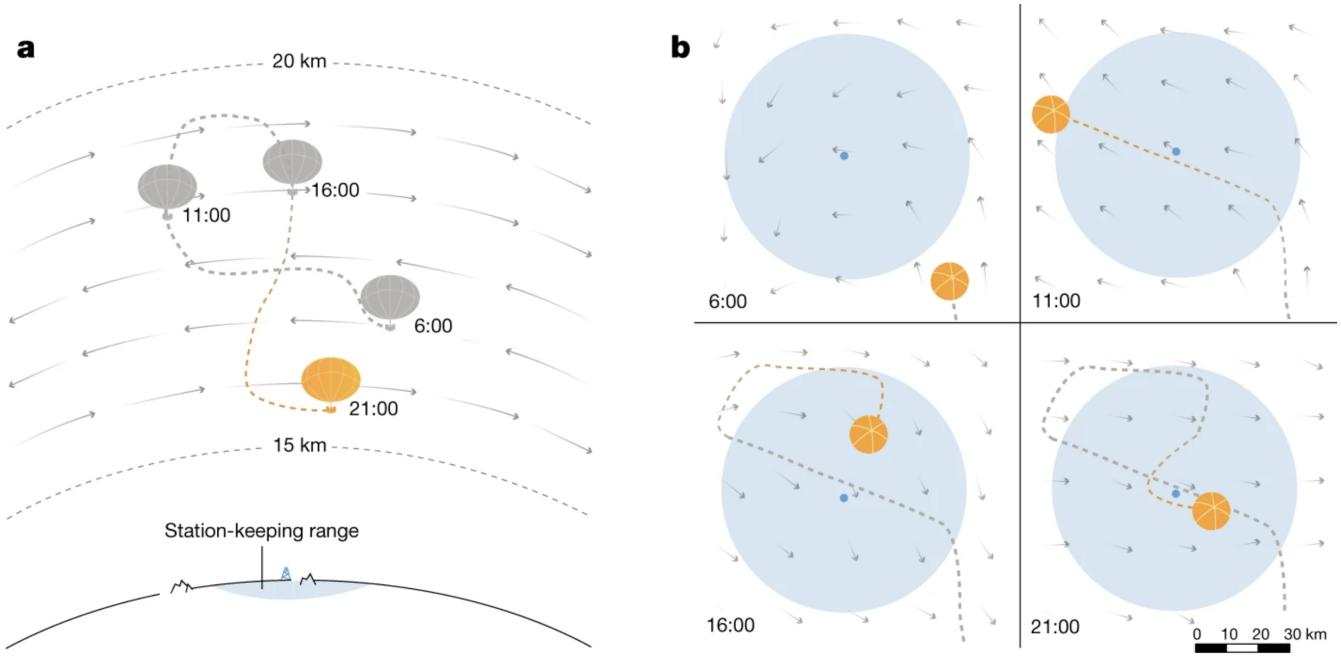
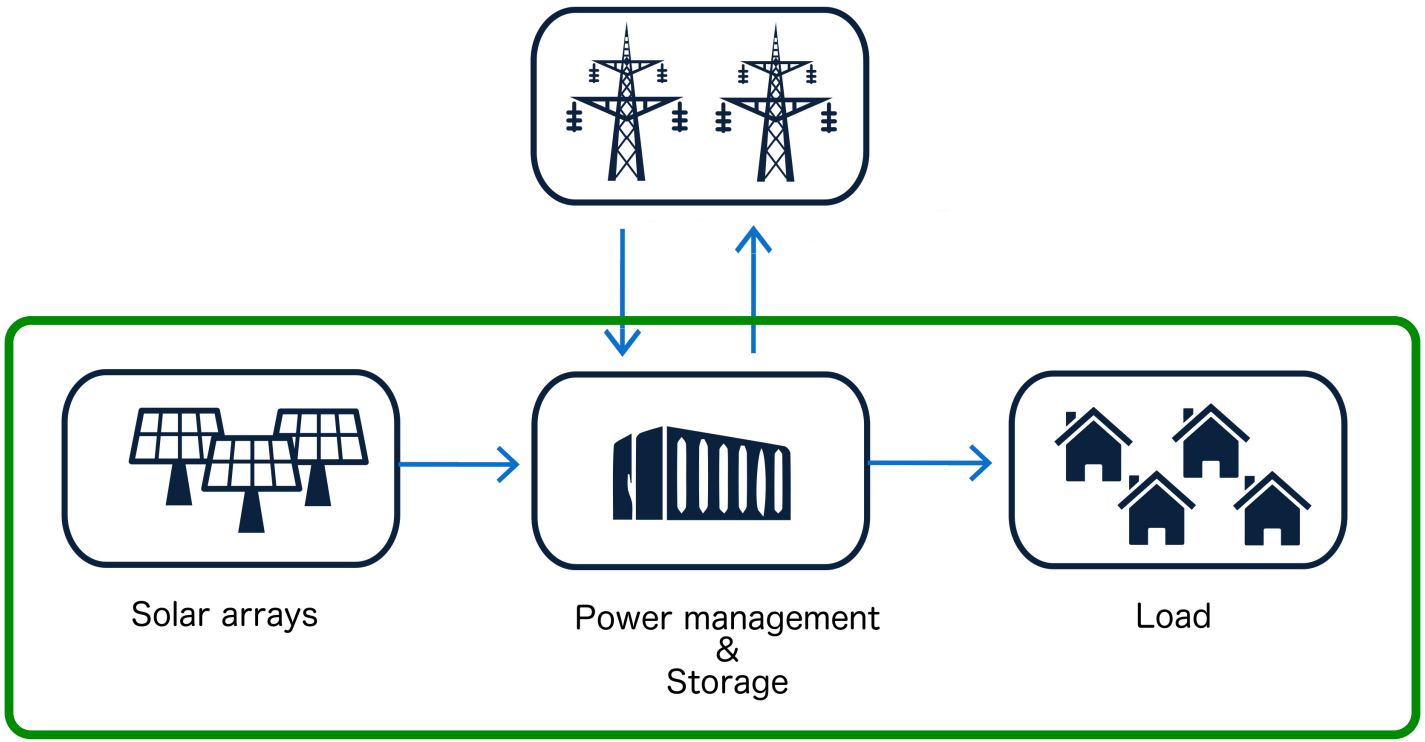


Figure: a, Schematic of a superpressure balloon navigating a wind field. The balloon remains close to its station by moving between winds at different altitudes. Its altitude range is indicated by the upper and lower dashed lines. b, The balloon's flight path, viewed from above. The station and its 50-km range are shown in light blue. Shaded arrows represent the wind field. The wind field constantly evolves, requiring the balloon to replan at regular intervals.

More details: *Autonomous navigation of stratospheric balloons using reinforcement learning*, M. G. Bellemare et al., 2020 (Nature).

Real-world application of deep RL: microgrid

A microgrid is an electrical system that includes multiple loads and distributed energy resources that can be operated in parallel with the broader utility grid or as an electrical island.

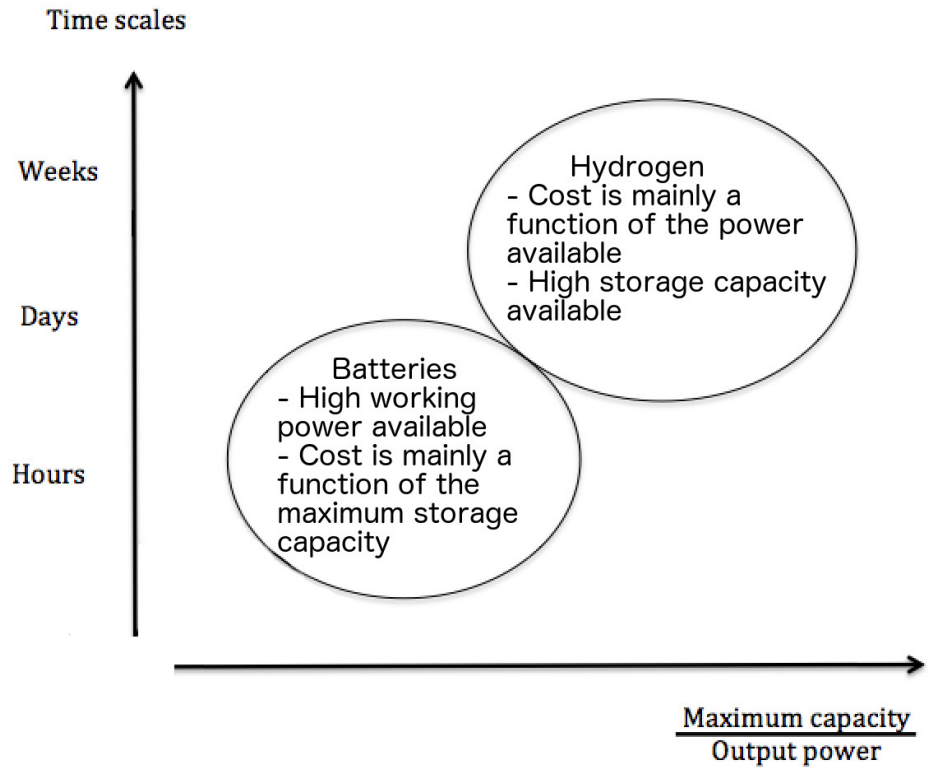


Microgrid

Microgrids and storage

There exist opportunities with microgrids featuring:

- A short term storage capacity (typically batteries),
- A long term storage capacity (e.g., hydrogen).



Structure of the Q-network

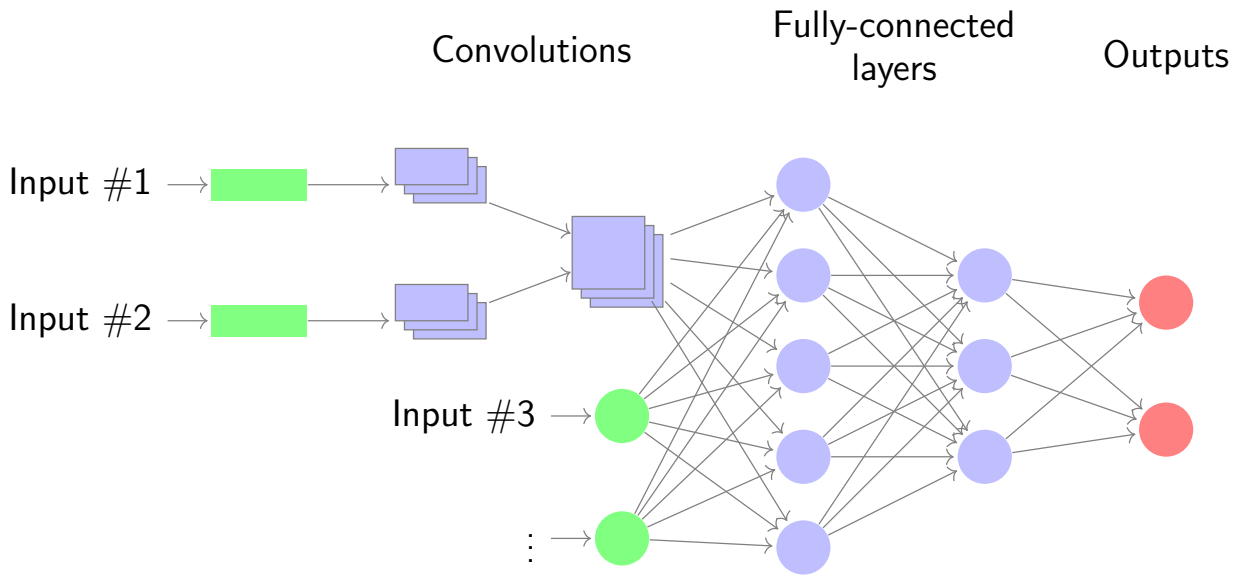


Figure: Sketch of the structure of the neural network architecture. The neural network processes the time series using a set of convolutional layers. The output of the convolutions and the other inputs are followed by fully-connected layers and the ouput layer. Architectures based on LSTMs instead of convolutions obtain similar results.

Results

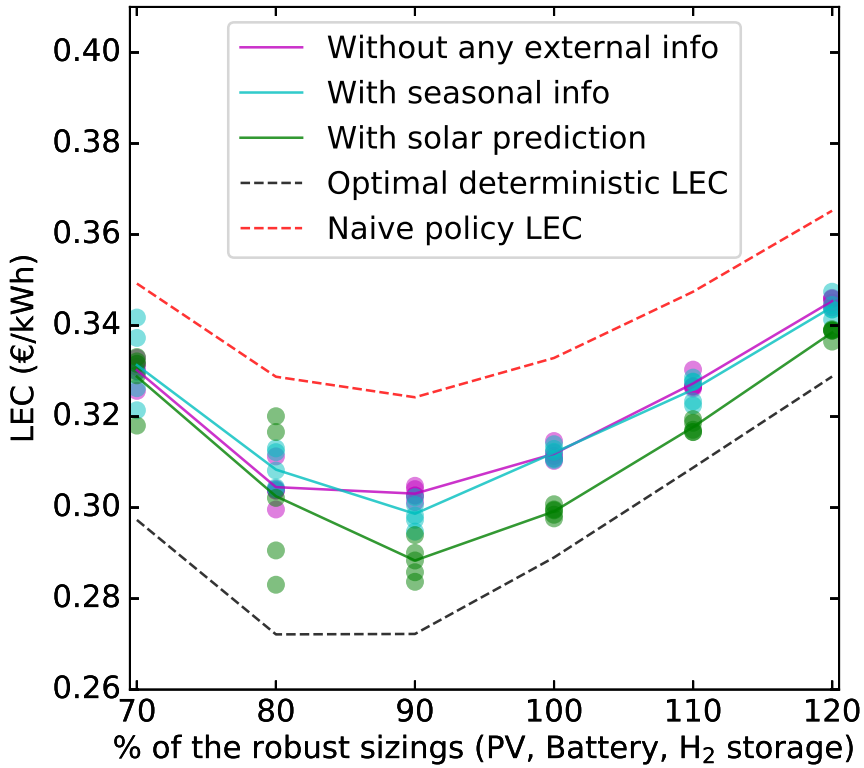


Figure: LEC on the test data function of the sizings of the microgrid.

More details: *Deep Reinforcement Learning Solutions for Energy Microgrids Management*, V. Francois-Lavet, D. Taralla, D. Ernst, R. Fonteneau, 2016 (EWRL).

Improving further generalization (links with some upcoming talks)

How to improve generalization?

We can improve generalization of RL thanks to the following elements:

- an **abstract representation** that discards non-essential features,
- the **objective function** (e.g., reward shaping, tuning the training discount factor) and
- the **learning algorithm** (type of function approximator and model-free vs model-based).

And of course, if possible:

- improve the dataset (exploration/exploitation dilemma in an online setting)

Combining model-free and model-based

Choice of the learning algorithm and function approximator selection

- The function approximator in deep learning characterizes how the features will be treated into higher levels of abstraction. A fortiori, it is related to feature selections (e.g., an attention mechanism), etc.
- Depending on the task, finding a performant function approximator is easier in either a model-free or a model-based approach. The choice of relying more on one or the other approach is thus also a crucial element to improve generalization.

Choice of the learning algorithm: a parallel with neurosciences

In cognitive science, there is a dichotomy between two modes of thoughts (*D. Kahneman. (2011). Thinking, Fast and Slow*) :

- a "System 1" that is fast and instinctive and
- a "System 2" that is slower and more logical.

Figure: System 1

Figure: System 2

In deep reinforcement, a similar dichotomy can be observed when we consider the model-free and the model-based approaches.

Planning

The dynamics for some sequence of actions is estimated recursively as follows for any t':

$$\hat{x}_{t'} = \begin{cases} e(s_t; \theta_e), & \text{if } t' = t \\ \hat{x}_{t'-1} + \tau(\hat{x}_{t'-1}, a_{t'-1}; \theta_\tau), & \text{if } t' > t \end{cases} \tag{3}$$

A set $\mathcal{A}^*$ of best potential actions is considered based on $Q(\hat{x}_t, a; \theta_Q)$ ($\mathcal{A}^* \subseteq \mathcal{A}$).

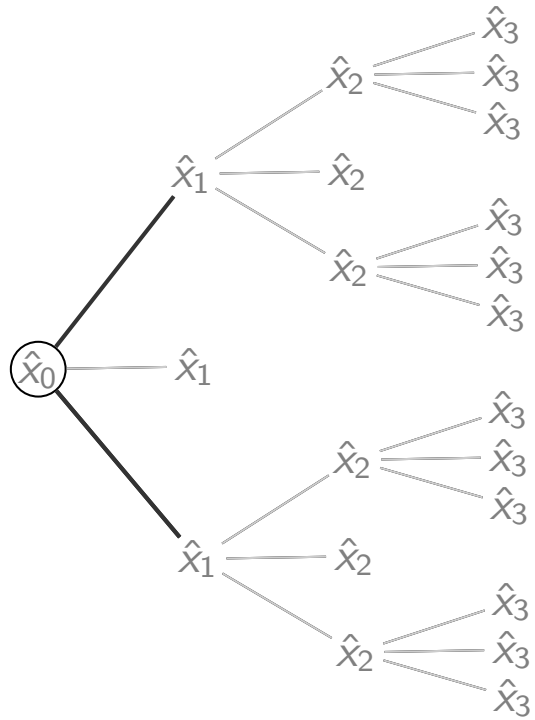


Figure: Expansion from current state representation x_0.

Planning

The dynamics for some sequence of actions is estimated recursively as follows for any t':

$$\hat{x}_{t'} = \begin{cases} e(s_t; \theta_e), & \text{if } t' = t \\ \hat{x}_{t'-1} + \tau(\hat{x}_{t'-1}, a_{t'-1}; \theta_\tau), & \text{if } t' > t \end{cases} \tag{3}$$

A set $\mathcal{A}^*$ of best potential actions is considered based on $Q(\hat{x}_t, a; \theta_Q)$ ($\mathcal{A}^* \subseteq \mathcal{A}$).

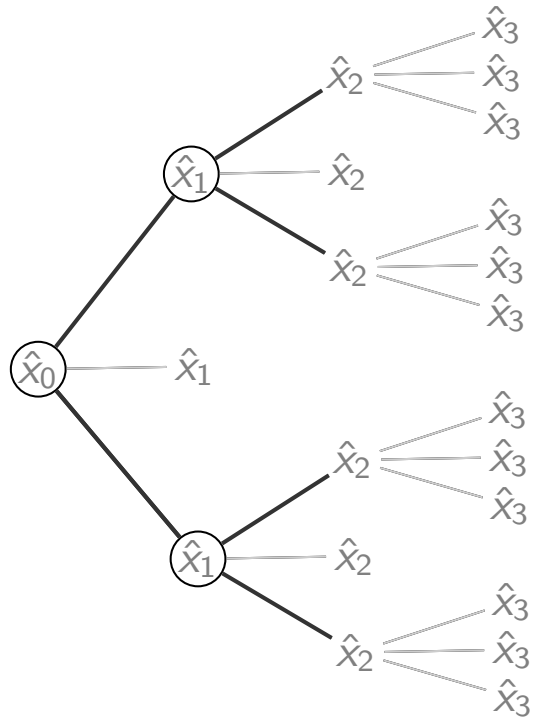


Figure: Expansion from current state representation x_0.

Planning

The dynamics for some sequence of actions is estimated recursively as follows for any t':

$$\hat{x}_{t'} = \begin{cases} e(s_t; \theta_e), & \text{if } t' = t \\ \hat{x}_{t'-1} + \tau(\hat{x}_{t'-1}, a_{t'-1}; \theta_\tau), & \text{if } t' > t \end{cases} \tag{3}$$

A set $\mathcal{A}^*$ of best potential actions is considered based on $Q(\hat{x}_t, a; \theta_Q)$ ($\mathcal{A}^* \subseteq \mathcal{A}$).

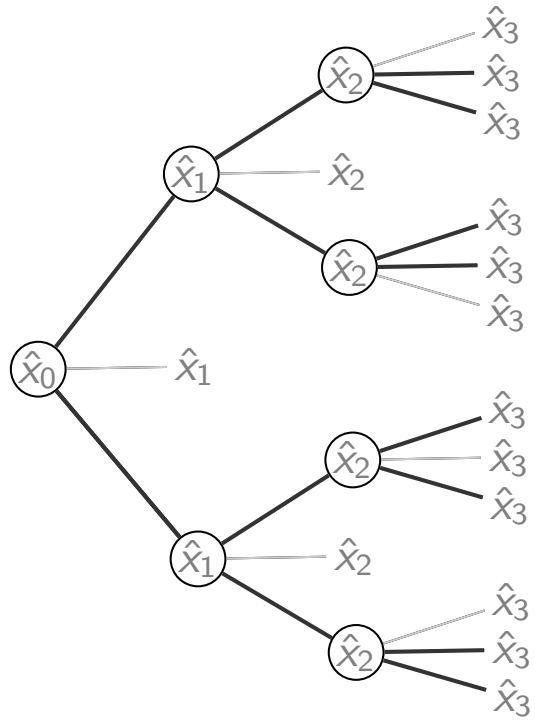


Figure: Expansion from current state representation x_0.

We define recursively the depth-d estimated expected return as

$$\hat{Q}^d(\hat{x}_t, a) = \begin{cases} \rho(\hat{s}_t, a; \theta_\rho) + g(\hat{x}_t, a; \theta_g) \max\limits_{a' \in \mathcal{A}^*} \hat{Q}^{d-1}(\hat{s}_{t+1}, a'), \\ \qquad\qquad\qquad\qquad\qquad\qquad \text{if } d > 0 \\ Q(\hat{s}_t, a; \theta_k), \text{ if } d = 0 \end{cases} \tag{4}$$

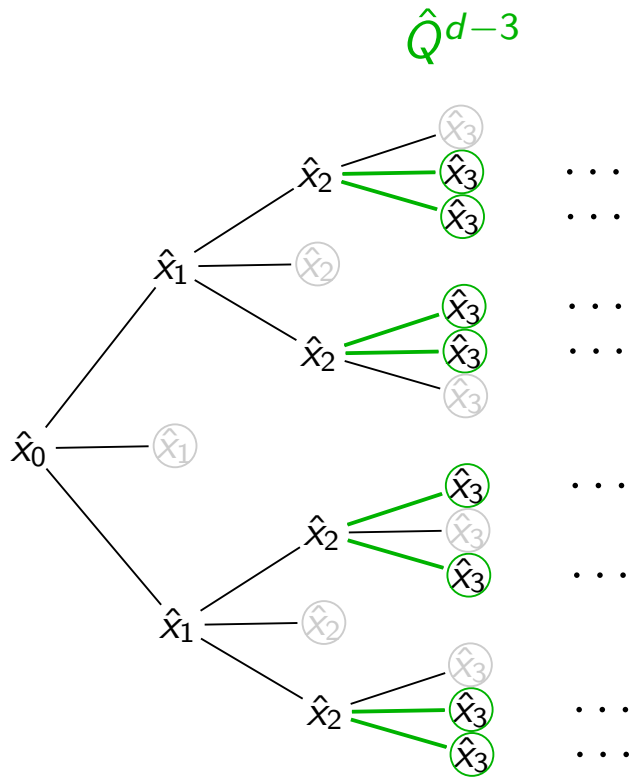


Figure: Backup.

We define recursively the depth-d estimated expected return as

$$\hat{Q}^d(\hat{x}_t, a) = \begin{cases} \rho(\hat{s}_t, a; \theta_\rho) + g(\hat{x}_t, a; \theta_g) \max\limits_{a' \in \mathcal{A}^*} \hat{Q}^{d-1}(\hat{s}_{t+1}, a'), \\ \qquad\qquad\qquad\qquad\qquad\qquad \text{if } d > 0 \\ Q(\hat{s}_t, a; \theta_k), \text{ if } d = 0 \end{cases} \tag{4}$$

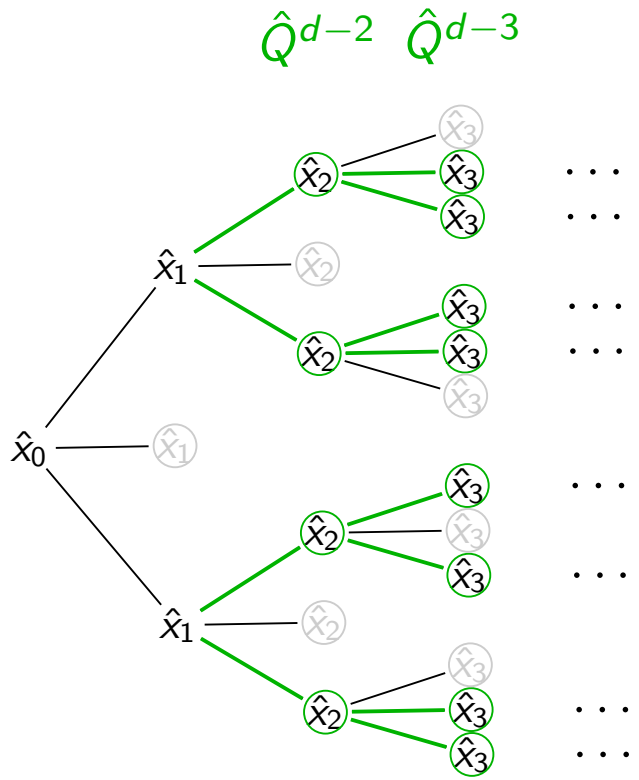


Figure: Backup.

We define recursively the depth-d estimated expected return as

$$\hat{Q}^d(\hat{x}_t, a) = \begin{cases} \rho(\hat{s}_t, a; \theta_\rho) + g(\hat{x}_t, a; \theta_g) \max\limits_{a' \in \mathcal{A}^*} \hat{Q}^{d-1}(\hat{s}_{t+1}, a'), \\ \qquad\qquad\qquad\qquad\qquad\qquad \text{if } d > 0 \\ Q(\hat{s}_t, a; \theta_k), \text{ if } d = 0 \end{cases} \tag{4}$$

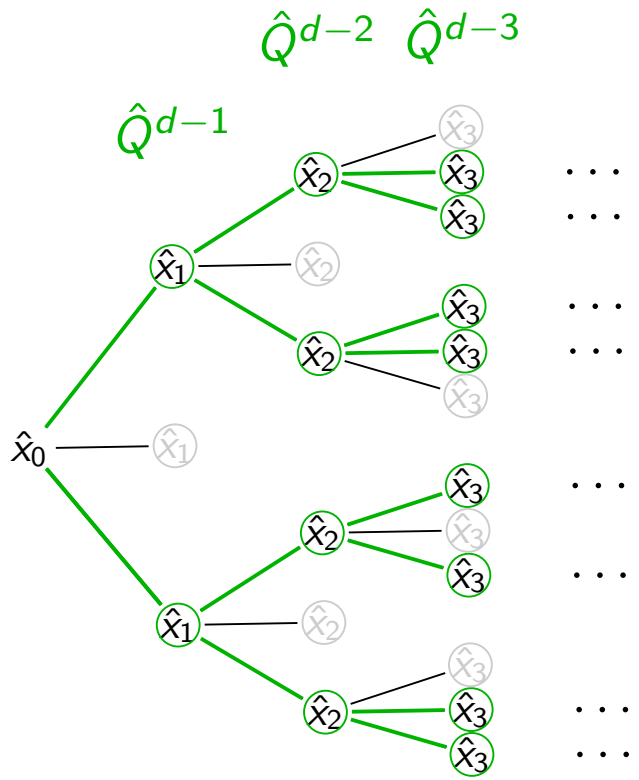


Figure: Backup.

We define recursively the depth-d estimated expected return as

$$\hat{Q}^d(\hat{x}_t, a) = \begin{cases} \rho(\hat{s}_t, a; \theta_\rho) + g(\hat{x}_t, a; \theta_g) \max\limits_{a' \in \mathcal{A}^*} \hat{Q}^{d-1}(\hat{s}_{t+1}, a'), \\ \qquad\qquad\qquad\qquad\qquad\qquad \text{if } d > 0 \\ Q(\hat{s}_t, a; \theta_k), \text{ if } d = 0 \end{cases} \tag{4}$$

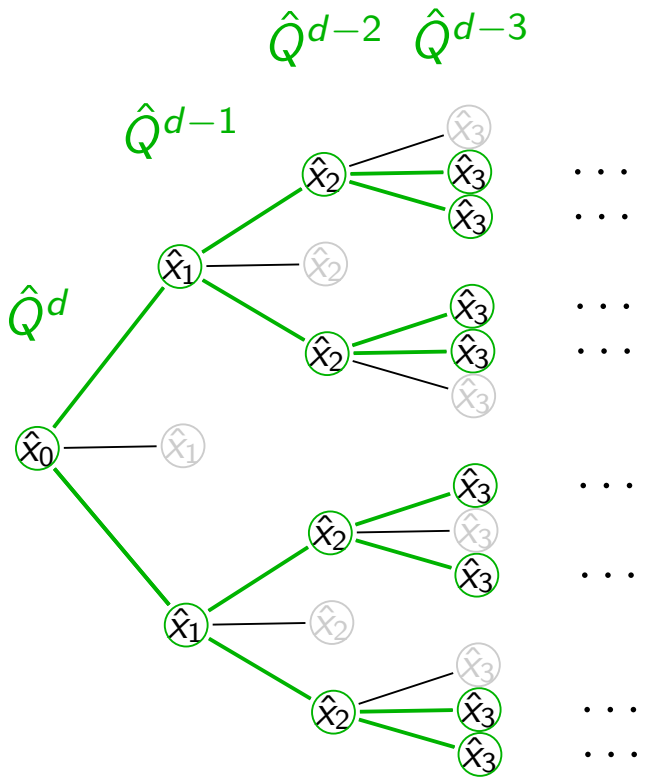


Figure: Backup.

Conclusions

Summary of the lecture

- Recall Q-learning in the tabular case
- Why function approximators are needed
- Q-learning with deep learning (DQN).
- Focus on different variants of model-free deep RL
- Some real-world examples using RL and function approximators
- Links with other talks in the week (e.g. model-based)

Questions?